МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ

з лабораторної роботи №4
з дисципліни “Професійний Python”
на тему “Професійне ООП та система типів Python”

Виконала
студентка 3 курсу
групи ФеП-33с
Корхова Марія Сергіївна

Перевірив
професор
Свелеба Сергій Андрійович

Львів 2026

Мета роботи: Сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів, протоколів, dataclass, properties, dunder methods, принципів SOLID та сучасної системи type hints.

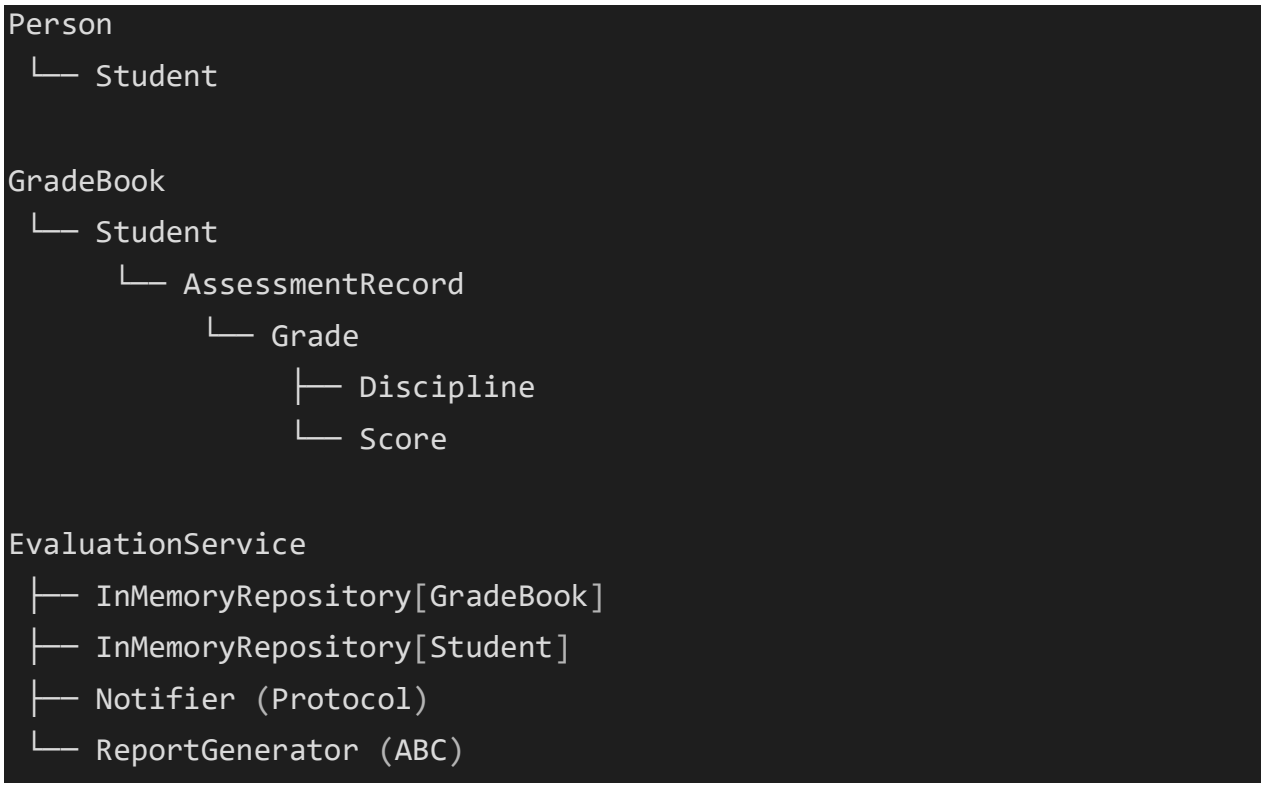
Варіант 7. Система оцінювання студентів Domain objects: • Student; • Discipline; • Grade; • AssessmentRecord; • GradeBook. Реалізувати: • додавання оцінок; • обчислення середнього, мінімального і максимального бала; • визначення успішності; • сортування студентів за середнім балом; • пошук студентів вище заданого порога; • repository; • reporting/notification abstractions.

Опис предметної області

Система оцінювання студентів. Дозволяє реєструвати студентів, додавати оцінки з дисциплін, обчислювати середній, мінімальний і максимальний бали, визначати загальну успішність (відсутність боргів), сортувати студентів за балом, фільтрувати їх за порогом та генерувати звіти через систему журналів (GradeBook).

Список domain entities

1. Person (базовий клас)
2. Student (доменний об'єкт студента)
3. Discipline (навчальна дисципліна)
4. Grade (оцінка з дисципліни)
5. AssessmentRecord (журнал оцінок окремого студента)
6. GradeBook (загальна відомість)
7. Score (Value Object для балів)
8. Class diagram



Пояснення inheritance (наслідування)

Наслідування використано для відображення зв'язку is-a (є чимось). Клас Student успадковує атрибути id та name від базового класу Person, розширюючи його специфічними для студента даними (group, email, record).

Пояснення composition (композиція)

Композиція відображає зв'язок has-a (має частину). Вона використана замість глибокого наслідування для більшої гнучкості:

- GradeBook містить колекцію об'єктів Student.
- Student агрегує AssessmentRecord.
- AssessmentRecord містить список Grade. Це дозволяє об'єктам бути незалежними компонентами.

Приклад encapsulation (інкапсуляції)

Інкапсуляція гарантує, що об'єкт сам контролює свій внутрішній стан. У класі AssessmentRecord список оцінок приховано у приватній змінній _grades. Додавання нових оцінок можливе лише через контрольований метод add_grade(), що запобігає некоректній зміні масиву ззовні.

```
class AssessmentRecord:
    def __init__(self) -> None:
        self._grades: list[Grade] = []

    def add_grade(self, grade: Grade) -> None:
        self._grades.append(grade)
```

Приклад property

Property надає інтерфейс атрибута до логіки, яка обчислюється динамічно. Замість зберігання статичного середнього бала, він вираховується "на льоту".

```
@property
    def average(self) -> float:
        if not self._grades:
            return 0.0
        return sum(g.score.value for g in self._grades) /
len(self._grades)
```

Приклад dataclass

Використання @dataclass автоматизує створення методів __init__, __repr__ та __eq__ для структур даних.

```
@dataclass
```

```
class Discipline:
    id: int
    name: str
    credits: float
```

Приклад immutable value object

Незмінний об'єкт-значення (Value Object) захищається параметром frozen=True. Об'єкт Score ініціалізується один раз і валідує свої межі.

```
@dataclass(frozen=True, slots=True)
class Score:
    value: float

    def __post_init__(self) -> None:
        if not (0.0 <= self.value <= 100.0):
            raise ValueError("Оцінка повинна бути в діапазоні від 0 до
100")
```

Приклад ABC

Абстрактні класи створюють строгий (номінальний) контракт, вимагаючи від нащадків реалізації абстрактних методів.

```
from abc import ABC, abstractmethod

class ReportGenerator(ABC):
    @abstractmethod
    def generate(self, grade_book: GradeBook) -> str:
        pass
```

Приклад Protocol

Протоколи забезпечують структурну типізацію (duck typing). Клас не зобов'язаний наслідувати протокол, достатньо мати відповідний метод `send`.

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None:
        ...
```

Щонайменше три dunder methods

Магічні методи інтегрують об'єкт у стандартну модель Python:

1. __str__ — для user-friendly форматування.
2. __len__ — для виклику len(record).
3. __iter__ — для ітерації for grade in record:.
4. __lt__ (less than) — для зручного сортування студентів за балом sorted(students).

Generic class

Узагальнений клас InMemoryRepository параметризується типом [T] і може зберігати будь-які типи, уникнувши дублювання коду.

```
class InMemoryRepository(Generic[T]):
    def add(self, item: T) -> None:
        self._items[item.id] = item
```

TypeVar

TypeVar дозволяє type checker-у зберігати зв'язок між входом і виходом функції/класу. bound=HasId обмежує типи лише тими, що мають поле id.

```
T = TypeVar("T", bound=HasId)
```

TypedDict

TypedDict типізує словники без створення об'єктів у runtime. Використовується для external API payloads.

```
class StudentPayload(TypedDict):
    id: int
    name: str
    group: str
    email: NotRequired[str]
```

Application service

EvaluationService виступає оркестратором. Він приймає залежності та керує бізнес-правилами (реєстрація, оцінювання, звітність).

Приклад dependency injection

Конкретні інфраструктурні об'єкти (репозиторії, нотифікатори) передаються в сервіс іззовні (через __init__), що полегшує заміну та тестування.

```
service = EvaluationService(
        gradebook_repo=InMemoryRepository[GradeBook](),
        student_repo=InMemoryRepository[Student](),
        notifier=ConsoleNotifier(),
        report_generator=SuccessReport()
    )
```

Пояснення щонайменше трьох SOLID principles

Single Responsibility Principle (SRP): Кожен клас має одну причину для зміни. InMemoryRepository відповідає лише за збереження даних, Score — за валідацію бала, SuccessReport — лише за форматування конкретного звіту.

Open/Closed Principle (OCP): Класи відкриті для розширення, але закриті для модифікації. Можна додати FailureReport як нову реалізацію ReportGenerator, не змінюючи код EvaluationService.

Dependency Inversion Principle (DIP): Високорівнева логіка EvaluationService залежить від абстракцій (наприклад, Notifier Protocol), а не від конкретного ConsoleNotifier.

Основний код program

Код моделі, репозиторіїв та сервісів (файли models.py, dto.py, protocols.py, repositories.py, services.py та main.py).

```
from typing import TypedDict, NotRequired

class StudentPayload(TypedDict):
    id: int
    name: str
    group: str
    email: NotRequired[str]
```

```
from domain_app.dto import StudentPayload
from domain_app.repositories import InMemoryRepository
from domain_app.services import ConsoleNotifier, SuccessReport,
EvaluationService
from domain_app.models import GradeBook, Discipline, Student

def main() -> None:
    student_repo = InMemoryRepository[Student]()
    gradebook_repo = InMemoryRepository[GradeBook]()

    gradebook = GradeBook(id=1, name="Відомість 2026")
    gradebook_repo.add(gradebook)
```

```
service = EvaluationService(
    gradebook_repo=gradebook_repo,
    student_repo=student_repo,
    notifier=ConsoleNotifier(),
    report_generator=SuccessReport()
)

payload_1: StudentPayload = {
    "id": 1,
    "name": "Марія Корхова",
    "group": "ФЕП-33",
    "email": "mariia.korkhova@lnu.edu.ua"
}
payload_2: StudentPayload = {
    "id": 2,
    "name": "Іван Петренко",
    "group": "ФЕП-33",
    "email": "ivan.p@example.com"
}

s1 = service.register_student(payload_1)
s2 = service.register_student(payload_2)

service.assign_student_to_gradebook(s1.id, gradebook.id)
service.assign_student_to_gradebook(s2.id, gradebook.id)

oop = Discipline(id=101, name="Професійне ООП", credits=4.0)
db = Discipline(id=102, name="Бази даних", credits=5.0)

service.evaluate_student(s1.id, oop, 98.5)
service.evaluate_student(s1.id, db, 92.0)

service.evaluate_student(s2.id, oop, 74.0)
```

```
    service.evaluate_student(s2.id, db, 45.0)

    print("\n--- Аналіз успішності (GradeBook) ---")
    for student in gradebook:
        print(student)
        print(f"  Мін: {student.record.min_score}, Макс:
{student.record.max_score}")
        print(f"  Успішність (без боргів): {'Так' if
student.record.is_successful else 'Ні'}\n")

    print("--- Сортування за середнім балом (Dunder __lt__) ---")
    for student in gradebook.sort_by_average():
        print(student)

    print("\n--- Фільтрація (вище 90 балів) ---")
    top_students = gradebook.filter_above_threshold(90.0)
    for student in top_students:
        print(f"Відмінник: {student.name}")

    print("\n--- Звіт ---")
    print(service.generate_success_report(gradebook.id))

if __name__ == "__main__":
    main()
```

```
from dataclasses import dataclass, field
from typing import Iterator
from domain_app.value_objects import Score

@dataclass(frozen=True)
class Discipline:
    id: int
```

```
    name: str
    credits: float

@dataclass
class Grade:
    discipline: Discipline
    score: Score

    def __str__(self) -> str:
        return f"{self.discipline.name}: {self.score}"

class AssessmentRecord:
    def __init__(self) -> None:
        self._grades: list[Grade] = []

    def add_grade(self, grade: Grade) -> None:
        self._grades.append(grade)

    @property
    def average(self) -> float:
        if not self._grades:
            return 0.0
        return sum(g.score.value for g in self._grades) /
len(self._grades)

    @property
    def min_score(self) -> float:
        if not self._grades:
            return 0.0
        return min(g.score.value for g in self._grades)

    @property
    def max_score(self) -> float:
```

```
        if not self._grades:
            return 0.0
        return max(g.score.value for g in self._grades)

    @property
    def is_successful(self) -> bool:
        if not self._grades:
            return False
        return self.min_score >= 51.0

    def __len__(self) -> int:
        return len(self._grades)

    def __iter__(self) -> Iterator[Grade]:
        return iter(self._grades)

@dataclass
class Person:
    id: int
    name: str

@dataclass
class Student(Person):
    group: str
    email: str
    record: AssessmentRecord = field(default_factory=AssessmentRecord)

    def __lt__(self, other: "Student") -> bool:
        return self.record.average < other.record.average

    def __str__(self) -> str:
        return f"[{self.id}] {self.name} ({self.group}) - Сер. бал:
{self.record.average:.2f}"
```

```
@dataclass
class GradeBook:
    id: int
    name: str
    _students: list[Student] = field(default_factory=list)

    def add_student(self, student: Student) -> None:
        self._students.append(student)

    def __iter__(self) -> Iterator[Student]:
        return iter(self._students)

    def sort_by_average(self, descending: bool = True) ->
list[Student]:
        return sorted(self._students, reverse=descending)

    def filter_above_threshold(self, threshold: float) ->
list[Student]:
        return [s for s in self._students if s.record.average >=
threshold]
```

```
from typing import Protocol, runtime_checkable

class HasId(Protocol):
    id: int

@runtime_checkable
class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None:
        ...
```

```python
from typing import Generic, TypeVar
from domain_app.protocols import HasId

T = TypeVar("T", bound=HasId)

class InMemoryRepository(Generic[T]):
    def __init__(self) -> None:
        self._items: dict[int, T] = {}

    def add(self, item: T) -> None:
        self._items[item.id] = item

    def get(self, item_id: int) -> T | None:
        return self._items.get(item_id)

    def all(self) -> list[T]:
        return list(self._items.values())

    def remove(self, item_id: int) -> bool:
        return self._items.pop(item_id, None) is not None
```

```python
from abc import ABC, abstractmethod
from domain_app.models import Student, Discipline, Grade, GradeBook
from domain_app.value_objects import Score
from domain_app.protocols import Notifier
from domain_app.repositories import InMemoryRepository
from domain_app.dto import StudentPayload

class ReportGenerator(ABC):
    @abstractmethod
    def generate(self, grade_book: GradeBook) -> str:
        pass
```

```python
class SuccessReport(ReportGenerator):
    def generate(self, grade_book: GradeBook) -> str:
        successful = [s for s in grade_book if s.record.is_successful]
        return f"Успішних студентів (без боргів): {len(successful)} з
{len(list(grade_book))}"

class ConsoleNotifier:
    def send(self, recipient: str, message: str) -> None:
        print(f"[Notifier -> {recipient}]: {message}")

class EvaluationService:
    def __init__(
        self,
        gradebook_repo: InMemoryRepository[GradeBook],
        student_repo: InMemoryRepository[Student],
        notifier: Notifier,
        report_generator: ReportGenerator,
    ) -> None:
        self._gradebook_repo = gradebook_repo
        self._student_repo = student_repo
        self._notifier = notifier
        self._report_generator = report_generator

    def register_student(self, payload: StudentPayload) -> Student:
        student = Student(
            id=payload["id"],
            name=payload["name"],
            group=payload["group"],
            email=payload.get("email", "unknown@domain.com")
        )
        self._student_repo.add(student)
        return student
```

```
    def assign_student_to_gradebook(self, student_id: int,
gradebook_id: int) -> None:
        student = self._student_repo.get(student_id)
        book = self._gradebook_repo.get(gradebook_id)
        if not student or not book:
            raise ValueError("Студент або журнал не знайдено")
        book.add_student(student)

    def evaluate_student(self, student_id: int, discipline:
Discipline, score_value: float) -> None:
        student = self._student_repo.get(student_id)
        if not student:
            raise ValueError("Студент не знайдений")

        grade = Grade(discipline=discipline, score=Score(score_value))
        student.record.add_grade(grade)
        self._notifier.send(student.email, f"Отримано оцінку
{score_value} з '{discipline.name}'")

    def generate_success_report(self, gradebook_id: int) -> str:
        book = self._gradebook_repo.get(gradebook_id)
        if not book:
            return "Журнал не знайдено."
        return self._report_generator.generate(book)
```

```
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class Score:
    value: float
```

```python
    def __post_init__(self) -> None:
        if not (0.0 <= self.value <= 100.0):
            raise ValueError("Оцінка повинна бути в діапазоні від 0 до 100")

    def __str__(self) -> str:
        return f"{self.value:.1f}"
```

Screenshot execution

```
(.venv) korkhovamariia@MariiaKorkhova:~/prof_Python/prof_Python/Lab_04/oop_project$ python3 src/domain_app/main.py
[Notifier -> mariia.korkhova@lnu.edu.ua]: Отримано оцінку 98.5 з 'Професійне ООП'
[Notifier -> mariia.korkhova@lnu.edu.ua]: Отримано оцінку 92.0 з 'Бази даних'
[Notifier -> ivan.p@example.com]: Отримано оцінку 74.0 з 'Професійне ООП'
[Notifier -> ivan.p@example.com]: Отримано оцінку 45.0 з 'Бази даних'

--- Аналіз успішності (GradeBook) ---
[1] Марія Корхова (ФЕП-33) - Сер. бал: 95.25
  Мін: 92.0, Макс: 98.5
  Успішність (без боргів): Так

[2] Іван Петренко (ФЕП-33) - Сер. бал: 59.50
  Мін: 45.0, Макс: 74.0
  Успішність (без боргів): Ні

--- Сортування за середнім балом (Dunder __lt__) ---
[1] Марія Корхова (ФЕП-33) - Сер. бал: 95.25
[2] Іван Петренко (ФЕП-33) - Сер. бал: 59.50

--- Фільтрація (вище 90 балів) ---
Відмінник: Марія Корхова

--- Звіт ---
Успішних студентів (без боргів): 1 з 2
```

Результат mypy

```
(.venv) korkhovamariia@MariiaKorkhova:~/prof_Python/prof_Python/Lab_04/oop_project$ mypy src
Success: no issues found in 8 source files
```

```
$ mypy src
Success: no issues found in 8 source files
```

Приклади знайдених type errors (Навмисний експеримент)

Було внесено наступні помилки до коду для перевірки статичного аналізатора:

1. **Помилка 1:** Передано неправильний тип параметра при ініціалізації:

```
student_id: int = "1"
# mypy: error: Incompatible types in assignment (expression has type
"str", variable has type "int")
```

2. **Помилка 2:** Невідповідність типу повернення абстракції:

```
class SuccessReport(ReportGenerator):
    def generate(self, grade_book: GradeBook) -> int:
        return 100
# mypy: error: Return type "int" of "generate" incompatible with return
type "str" in supertype "ReportGenerator"
```

3. **Помилка 3:** Помилка Generic-репозиторію:

```
repo: InMemoryRepository[GradeBook] = InMemoryRepository()
repo.add(Student(id=1, name="...", group="...", email="..."))
# mypy: error: Argument 1 to "add" of "InMemoryRepository" has
incompatible type "Student"; expected "GradeBook"
```

Виправлений результат type checking

Після усунення внесених помилок і повернення до початкової коректної типізації `StudentPayload` та `GradeBook`, mypy знову показав 0 помилок:

```
(.venv) korkhovamariia@MariiaKorkhova:~/prof_Python/prof_Python/Lab_04/oop_project$ mypy src
Success: no issues found in 8 source files
```

```
$ mypy src
Success: no issues found in 8 source files
```

Висновок

У лабораторній роботі №4 було здійснено перехід до побудови повноцінної об'єктної моделі предметної області. В результаті роботи створено типізовану систему, де кожна сутність має чітку зону відповідальності, а залежності контролюються через Dependency Injection. Композиція забезпечила побудову гнучкої поведінки, наслідування використано для реальних відношень "є чимось", а ABC та Protocol сформували чисті абстракції. Система типів Python (TypeVar, Generic, TypedDict) та аналізатор `mypy` дозволили інтегрувати жорсткий контроль типів, виявляючи помилки архітектури ще до запуску програми.